

Entropie croisée binaire

Binary Cross-Entropy · Log-Loss

Définition

La fonction de coût de la régression logistique binaire s'appelle entropie croisée binaire (binary cross-entropy ou log-loss).

Elle mesure à quel point les probabilités prédites par le modèle sont bonnes.

Elle donne une faible pénalité si le modèle prédit avec confiance la bonne classe, et une très forte pénalité s'il se trompe avec confiance.

1. La formule

Le modèle prédit pour chaque observation i une probabilité :

Probabilité prédite

$$\hat{y}_i = \sigma(\beta_0 + \sum \beta_j x_{ij})$$

où $\hat{y}_i$ est la probabilité prédite que l'observation appartienne à la classe 1.

La fonction de coût mesure ensuite l'erreur globale sur toutes les n observations :

Entropie croisée binaire $J(\beta)$

$$J(\beta) = -(1/n) \cdot \sum [y_i \cdot \log(\hat{y}_i) + (1 - y_i) \cdot \log(1 - \hat{y}_i)]$$

n = nombre d'observations · $y_i \in \{0,1\}$ = vraie classe · $\hat{y}_i$ = probabilité prédite

2. Pourquoi deux termes ?

En classification binaire, chaque observation est soit classe 1, soit classe 0. La formule s'adapte automatiquement grâce à la structure des deux termes.

Cas 1 — Si $y_i = 1$ (vraie classe positive)

Le terme $(1 - y_i) = 0$ disparaît. Il reste :

$$y_i \cdot \log(\hat{y}_i) + (1 - y_i) \cdot \log(1 - \hat{y}_i) = \log(\hat{y}_i) \rightarrow \text{perte} = -\log(\hat{y}_i)$$

Pour minimiser cette perte, on veut $\hat{y}_i$ proche de 1.

Cas 2 — Si $y_i = 0$ (vraie classe négative)

Le terme $y_i = 0$ disparaît. Il reste :

$$y_i \cdot \log(\hat{y}_i) + (1-y_i) \cdot \log(1-\hat{y}_i) = \log(1-\hat{y}_i) \rightarrow \text{perte} \\ = -\log(1-\hat{y}_i)$$

Pour minimiser cette perte, on veut $\hat{y}_i$ proche de 0.

💡 Résumé des deux cas

Si $y = 1 \rightarrow \text{perte} = -\log(\hat{y})$ on veut $\hat{y}$ proche de 1

Si $y = 0 \rightarrow \text{perte} = -\log(1 - \hat{y})$ on veut $\hat{y}$ proche de 0

Dans les deux cas, une prédiction confiante et correcte donne une perte proche de 0.

3. Exemples numériques

Voici des exemples concrets pour comprendre comment la perte varie selon la probabilité prédite.

Cas $y_i = 1$ — le patient est vraiment malade

Vraie classe y_i	Probabilité prédite $\hat{y}_i$	Perte $-\log(\hat{y})$	Interprétation
$y = 1$	$\hat{y} = 0.90$	0.105	Excellent — confiant et correct
$y = 1$	$\hat{y} = 0.60$	0.511	Correct mais peu confiant
$y = 1$	$\hat{y} = 0.01$	4.605	Catastrophique — confiant dans la mauvaise classe

Cas $y_i = 0$ — le patient est vraiment sain

Vraie classe y_i	Probabilité prédite $\hat{y}_i$	Perte $-\log(1-\hat{y})$	Interprétation
$y = 0$	$\hat{y} = 0.10$	0.105	Excellent — prédit bien la classe 0
$y = 0$	$\hat{y} = 0.50$	0.693	Incertitude totale
$y = 0$	$\hat{y} = 0.80$	1.609	Mauvais — confiant dans la mauvaise classe

4. Pourquoi un logarithme ?

Le logarithme crée une asymétrie très utile : une prédiction confiante et correcte coûte presque rien ; une prédiction confiante et fausse coûte énormément.

Probabilité prédite	$\log(\hat{y})$	Perte $-\log(\hat{y})$	Effet
$\hat{y} = 0.99$	≈ -0.010	≈ 0.010	Quasi nulle — très bonne prédiction
$\hat{y} = 0.80$	≈ -0.223	≈ 0.223	Faible
$\hat{y} = 0.50$	≈ -0.693	≈ 0.693	Modérée — incertitude maximale
$\hat{y} = 0.20$	≈ -1.609	≈ 1.609	Grande

$\hat{y} = 0.01$	≈ -4.605	≈ 4.605	Énorme — erreur confiante très pénalisée
------------------	------------------	-----------------	--

Conclusion : Le logarithme rend le modèle très sensible aux erreurs confiantes. C'est exactement ce qu'on veut en classification probabiliste.

5. Pourquoi le signe moins ?

Les logarithmes de nombres entre 0 et 1 sont toujours négatifs : $\log(0.9) < 0$, $\log(0.5) < 0$, $\log(0.1) < 0$.
On place donc un **signe moins devant toute la formule** pour obtenir une quantité positive à minimiser.

6. Pourquoi diviser par n ?

La division par **n** transforme la somme en une **moyenne par observation**. Cela présente deux avantages :

- $J(\beta)$ ne dépend pas du nombre d'observations — on peut comparer des modèles sur des datasets de tailles différentes.
- La perte reste dans un intervalle interprétable, indépendamment de la taille du dataset.

7. Intuition globale

Cette fonction récompense les bonnes probabilités confiantes et pénalise très fortement les mauvaises prédictions confiantes.

Scénario	Prédiction	Perte	Jugement
Vrai positif	$\hat{y} = 0.95$ (confiant, correct)	≈ 0.05	Très bien
Vrai positif	$\hat{y} = 0.55$ (hésitant, correct)	≈ 0.60	Correct mais peut mieux faire
Vrai positif	$\hat{y} = 0.01$ (confiant, faux)	≈ 4.61	Très grave
Vrai négatif	$\hat{y} = 0.05$ (confiant, correct)	≈ 0.05	Très bien
Vrai négatif	$\hat{y} = 0.80$ (confiant, faux)	≈ 1.61	Mauvais

8. Que fait l'algorithme d'optimisation ?

L'algorithme (descente de gradient) cherche les coefficients β qui minimisent $J(\beta)$:

- **Les vrais 1** doivent recevoir des probabilités proches de 1.
- **Les vrais 0** doivent recevoir des probabilités proches de 0.

Propriété clé : convexité

L'entropie croisée est convexe en β — contrairement à la MSE appliquée à la régression logistique.

Cela garantit que la descente de gradient converge toujours vers l'optimum global : pas de minimum local.

C'est une des raisons fondamentales pour lesquelles on utilise cette fonction plutôt que la MSE pour la classification.

9. Lien avec la vraisemblance

Cette formule n'est pas choisie arbitrairement. Il existe une justification statistique fondamentale.

Théorème fondamental

**Minimiser $J(\beta)$ = Maximiser la vraisemblance du modèle
logistique**

Autrement dit, on choisit les paramètres β qui rendent les données observées les plus probables selon le modèle. C'est le principe du Maximum de Vraisemblance (MLE).

☑ Récapitulatif complet

$J(\beta)$: Entropie croisée binaire — mesure la qualité des probabilités prédites

Signe - : Pour obtenir une perte positive à minimiser ($\log < 0$ sur $]0,1[$)

$\div n$: Moyenne sur toutes les observations

$y \cdot \log(\hat{y})$: Actif quand $y=1$ · pénalise $\hat{y}$ loin de 1

$(1-y) \cdot \log(1-\hat{y})$: Actif quand $y=0$ · pénalise $\hat{y}$ loin de 0

Logarithme : Pénalité forte si confiant et faux · faible si confiant et correct

Convexité : Convergence garantie vers l'optimum global

MLE : Minimiser $J(\beta)$ = Maximiser la vraisemblance

Tableau de référence — valeurs de perte

y_i	$\hat{y}_i$	Perte	Interprétation
1	0.99	0.010	Parfait — très confiant, très juste
1	0.90	0.105	Excellent
1	0.70	0.357	Bien
1	0.50	0.693	Incertain — perte modérée
1	0.30	1.204	Mauvais
1	0.10	2.303	Très mauvais
1	0.01	4.605	Catastrophique
0	0.01	0.010	Parfait — prédit bien la classe 0
0	0.10	0.105	Excellent
0	0.30	0.357	Bien
0	0.50	0.693	Incertain
0	0.70	1.204	Mauvais
0	0.90	2.303	Très mauvais
0	0.99	4.605	Catastrophique

💬 Phrase à retenir

« L'entropie croisée binaire mesure la qualité des probabilités prédites en classification binaire : elle pénalise fortement les prédictions très confiantes mais fausses. »